

DAY - 30: "From Zero to OOP Hero with JavaScript ES6 Classes"



Created by: Neeraj | [LinkedIn: neeraj-kumar1904](#)  | [X: @_19_neeraj](#)  | [GitHub: Neeraj05042001](#)  |

Table of Contents

1. [What is a Class?](#)
 2. [ES6 Classes - Syntax](#)
 3. [Initialize Objects - "this" in Classes](#)
 4. [The Type of Class](#)
 5. [Class as Expression](#)
 6. [Class Fields](#)
 7. [Getters and Setters](#)
 8. [Static Properties](#)
 9. [Private and Public Fields](#)
 10. [Extending Classes](#)
 11. [OOP Concepts](#)
 12. [Interview Questions](#)
 13. [Practice Tasks](#)
-

Introduction to ES6 Classes

Definition: ES6 Classes are a syntactic sugar over JavaScript's prototype-based inheritance system, providing a cleaner and more intuitive way to create objects and implement object-oriented programming concepts.

Why Classes Matter

- **Cleaner Syntax:** More readable than prototype-based approach
- **OOP Support:** Enables proper object-oriented programming
- **Industry Standard:** Widely used in modern JavaScript frameworks

- **Better Organization:** Helps structure large applications

Knowledge Check

- ▶  What is the main advantage of ES6 classes over prototype-based inheritance?

What is a Class?

Definition

A **Class** is a blueprint or template for creating objects with similar properties and methods. It defines the structure and behavior that objects created from it will have.

Think of it like a cookie cutter - it defines the shape, but each cookie (object) is unique.

Basic Structure

```
class ClassName {  
    // Properties and methods go here  
}
```

Key Points:

-  Classes are introduced in ES6 (ES2015)
-  They provide a cleaner syntax for creating objects
-  They're syntactic sugar over JavaScript's existing prototype-based inheritance
-  Classes act as factories for creating multiple objects with similar properties and methods

Real-Life Analogy

Think of a class like a **cookie cutter** :

- The cookie cutter (class) defines the shape
- Each cookie (object) made from it has the same shape but can have different flavors
- You can make many cookies (objects) from one cutter (class)

Knowledge Check:

- ▶ Q: What is a Class?
 - ▶ Q: How is a class similar to a blueprint?
-

ES6 Classes - Syntax

Basic Syntax:

```
class ClassName {  
  constructor(parameters) {  
    // Initialize properties  
  }  
  
  method1() {  
    // Method implementation  
  }  
  
  method2() {  
    // Method implementation  
  }  
}
```

Example:

Basic Class Declaration

```
class Person {  
  constructor(name, age) {  
    this.name = name;  
    this.age = age;  
  }  
  
  greet() {  
    return `Hello, I'm ${this.name}`;  
  }  
}  
  
// Creating objects (instances)  
const person1 = new Person("Alice", 25);  
const person2 = new Person("Bob", 30);  
  
console.log(person1.greet()); // "Hello, I'm Alice"
```

Key Components:

- `class` keyword: Declares a class. `class ClassName`
- `constructor()` : Special method that runs when creating new instances
- Methods: Functions defined inside the class
- Properties: Variables that belong to the class

- `new` keyword: Creates a new instance of the class

► 🎯 Knowledge Check: Class Syntax

🔧 Constructor and "this" Keyword

🎯 Constructor Method

Definition: The constructor is a special method that runs automatically when a new object is created from the class.

```
class Car {  
  constructor(brand, model, year) {  
    this.brand = brand;    // this refers to the current instance  
    this.model = model;  
    this.year = year;  
    this.isRunning = false;  
  }  
  
  start() {  
    this.isRunning = true;  
    return `${this.brand} ${this.model} is now running!`;  
  }  
}  
  
const myCar = new Car("Toyota", "Camry", 2023);  
console.log(myCar.start()); // "Toyota Camry is now running!"
```

🔍 Understanding "this"

- In Constructor: `this` refers to the new object being created
- In Methods: `this` refers to the object that called the method
- Binding: `this` is automatically bound to the instance

🎯 Example:

```
class Car {  
  constructor(model) {  
    this.model = model; // 'this' refers to the current instance  
  }  
  
  printModel() {  
    console.log(this.model); // Accessing the instance property  
  }  
}
```

```
const bmwCar = new Car("BMW");  
bmwCar.printModel(); // Output: BMW
```

How it Works:

1. `new Car("BMW")` creates a new instance
2. `this.model = model` assigns "BMW" to the instance's model property
3. `this.model` in methods refers to that specific instance's property

►  Knowledge Check: "this" in Classes

The Type of Class

Understanding Class Types:

```
class Car {  
  constructor(model) {  
    this.model = model;  
  }  
}  
  
console.log(typeof Car); // "function"  
console.log(Car.prototype);  
console.log(Car.prototype.constructor);  
console.log(Car === Car.prototype.constructor); // true
```

Key Insights:

- Classes are functions under the hood
- They have a prototype property
- The constructor property points back to the class itself
- Classes are essentially syntactic sugar over function constructors

►  Knowledge Check: Class Types

Class as Expression

Anonymous Class Expression:

```
const Employee = class {  
  welcome() {  
    console.log("Hello Employee");  
  }  
}  
  
const emp = new Employee();  
emp.welcome(); // Output: Hello Employee
```

Named Class Expression:

```
const Dept = class Department {  
  welcome() {  
    console.log("Welcome to Department");  
    console.debug(Department); // Name accessible inside  
  }  
}  
  
const d = new Dept();  
d.welcome();
```

Key Points:

- Classes can be defined as expressions
- Can be anonymous or named
- Named class expressions: name is only accessible inside the class
- Useful for dynamic class creation

► Knowledge Check: Class Expressions

Class Fields

Definition:

Class fields allow you to define properties directly in the class body without the constructor.

Example:

```
class Phone {  
  brand = "Apple"; // Class field  
  
  make() {  
    console.log(this.brand);  
  }  
}
```

```
    }  
  }  
  
  const phone = new Phone();  
  console.log(phone.brand); // "Apple"  
  phone.make(); // "Apple"
```

Comparison:

```
// Without class fields (traditional way)  
class Phone1 {  
  constructor() {  
    this.brand = "Apple";  
  }  
}  
  
// With class fields (modern way)  
class Phone2 {  
  brand = "Apple";  
}
```

Benefits:

- Cleaner syntax
- No need to use constructor for simple property initialization
- More readable code

- ▶  Knowledge Check: Class Fields
- ▶  What type does a class return when checked with typeof?

Getters and Setters

Definition:

Getters and setters are special methods that allow you to define how properties are accessed and modified.

Example:

```
class Animal {  
  constructor(name) {  
    this.name = name; // This will use the setter  
  }  
}
```

```
get name() {  
    return this._name; // getter  
}  
  
set name(value) {  
    if (!value) {  
        console.warn("Need a name");  
        return;  
    }  
    this._name = value; // setter with validation  
}  
}  
  
const animal = new Animal("Tiger");  
console.log(animal.name); // Uses getter  
animal.name = ""; // Uses setter, shows warning
```

🔑 Key Points:

- `get` keyword defines a getter method
- `set` keyword defines a setter method
- Allows data validation and processing
- Properties accessed like regular properties, not method calls
- Common convention: use underscore prefix for private-like properties (`_name`)

🎯 Benefits of Getters/Setters

- Validation: Check data before setting
- Computed Properties: Calculate values on the fly
- Encapsulation: Control access to internal data
- Backward Compatibility: Change internal implementation without breaking code

📋 Knowledge Check

- ▶ 🤔 How do you access a getter method?
- ▶ 💬 Knowledge Check: Getters and Setters

⚡ Static Properties

📖 Definition:

Static methods and properties belong to the class itself, not to instances of the class. They can be called without creating an instance.

Basic Example:

```
class MyClass {
    static staticMethod() {
        console.log(this); // 'this' refers to the class itself
    }
}

MyClass.staticMethod(); // Called on class, not instance
```

Static Methods

```
class MathUtils {
    static PI = 3.14159;

    static add(a, b) {
        return a + b;
    }

    static multiply(a, b) {
        return a * b;
    }

    static circleArea(radius) {
        return this.PI * radius ** 2;
    }
}

// Called on the class, not on instances
console.log(MathUtils.add(5, 3)); // 8
console.log(MathUtils.circleArea(5)); // 78.53975
```

Static Properties

```
class User {
    static userCount = 0;
    static maxUsers = 100;

    constructor(name) {
        if (User.userCount >= User.maxUsers) {
            throw new Error("Maximum users reached");
        }

        this.name = name;
    }
}
```

```

        User.userCount++; // Increment static property
    }

    static getTotalUsers() {
        return User.userCount;
    }
}

const user1 = new User("Alice");
const user2 = new User("Bob");
console.log(User.getTotalUsers()); // 2

```

Advanced Example:

```

class User {
    constructor(name, email) {
        this.name = name;
        this.email = email;
    }

    greet() {
        console.log(`Hi, I'm ${this.name}`);
    }

    // Static utility method
    static isValidEmail(email) {
        return email.includes("@") && email.includes(".");
    }

    // Static factory method
    static createGuest() {
        return new User("Guest", "guest@example.com");
    }
}

// Using static methods without creating instances
console.log(User.isValidEmail("john@example.com")); // true
const guestUser = User.createGuest();
guestUser.greet(); // Hi, I'm Guest

```

When to Use Static

- Utility functions: Helper methods related to the class
- Factory methods: Alternative ways to create instances
- Constants: Class-level constants
- Validation methods: Check data without creating instances



Private and Public Fields



Definition:

- Public: Accessible from anywhere (default behavior)
- Private: Accessible only from within the class (prefixed with `#`)
- Protected: By convention, prefixed with `_` (not truly private)



Comprehensive Example:

```
class WashingMachine {
  // Public field
  brand;

  // Private fields (truly private)
  #powerStatus = false;
  #currentCycle = null;

  // Protected field (naming convention only)
  _log = [];

  constructor(brand) {
    this.brand = brand;
  }

  // Public method
  start(cycle) {
    if (!this.#powerStatus) {
      this.#turnOn();
    }
    this.#currentCycle = cycle;
    console.log(`Starting ${cycle} cycle...`);
    this.#spin();
    this.#drain();
    this._log.push(`Cycle ${cycle} completed.`);
    this.stop();
  }

  // Public method
  stop() {
    console.log("Washing machine stopped.");
    this.#turnOff();
  }

  // Private methods
  #turnOn() {
    this.#powerStatus = true;
    console.log("Power ON");
  }
}
```

```

    }

    #turnOff() {
        this.#powerStatus = false;
        console.log("Power OFF");
    }

    #spin() {
        console.log("Spinning...");
    }

    #drain() {
        console.log("Draining...");
    }

    // Protected method (convention)
    _showLog() {
        console.log("Internal Logs:", this._log);
    }
}

const lgWasher = new WashingMachine("LG");
lgWasher.start("Quick Wash");
console.log(lgWasher.brand); // ✅ Works - public
// console.log(lgWasher.#powerStatus); // ❌ SyntaxError - private
lgWasher._showLog(); // ⚠️ Works but not recommended - protected

```

Access Levels Summary:

Type	Symbol	Accessibility	Example
Public	none	Everywhere	<code>brand</code>
Private	<code>#</code>	Class only	<code>#powerStatus</code>
Protected	<code>_</code>	Convention only	<code>_log</code>

Benefits of Private Fields

- Encapsulation: Hide internal implementation
- Security: Prevent external tampering
- Maintainability: Internal changes don't break external code
- Clear Interface: Shows what's meant to be used publicly

►  Knowledge Check: Private and Public

Extending Classes

Definition:

Class inheritance allows you to create a new class based on an existing class, inheriting its properties and methods while adding new functionality.

Comprehensive Example:

```
// Parent class (Base class)
class Human {
  species = "Homo Sapiens";

  constructor(name, age) {
    this.name = name;
    this.age = age;
  }

  introduce() {
    console.log(`Hi, I'm ${this.name}, and I'm ${this.age} years old.`);
  }

  sleep() {
    console.log(`${this.name} is sleeping.`);
  }
}

// Child class (Derived class)
class Student extends Human {
  species = "Homo Sapiens (Student)"; // Override field

  constructor(name, age, grade) {
    super(name, age); // Call parent constructor
    this.grade = grade; // Add new property
  }

  // Method overriding
  introduce() {
    console.log(
      `Hey! I'm ${this.name}, ${this.age} years old and I study in grade ${this.grade}.`
    );
  }

  // New method
  study() {
    console.log(`${this.name} is studying...`);
  }
}

class Teacher extends Human {
  constructor(name, age, subject) {
    super(name, age);
    this.subject = subject;
  }
}
```

```

    }

    introduce() {
        console.log(`Hello, I'm ${this.name}, a ${this.subject} teacher.`);
    }

    teach() {
        console.log(`${this.name} is teaching ${this.subject}.`);
    }
}

// Usage
const alice = new Student("Alice", 14, 9);
const bob = new Teacher("Bob", 35, "Mathematics");

alice.introduce(); // Overridden method
bob.introduce();   // Overridden method
alice.sleep();      // Inherited method
bob.sleep();        // Inherited method

```

🔑 Key Concepts:

- `extends` : Creates inheritance relationship
- `super()` : Calls parent constructor or methods
- Method Overriding: Child class can redefine parent methods
- Method Addition: Child class can add new methods

▶ 🗨 Knowledge Check: Extending Classes

🏛 OOP Concepts

1. 📦 Abstraction

📖 Definition: Hiding complex implementation details while showing only essential features.

🎯 Example:

```

class PaymentProcessor {
    processPayment(amount, method) {
        // Abstract the complex payment logic
        this.#validatePayment(amount);
        this.#connectToGateway();
        this.#processTransaction(amount, method);
        return this.#getConfirmation();
    }
}

```

```
// Hidden complex methods
#validatePayment(amount) { /* validation logic */ }
#connectToGateway() { /* gateway connection */ }
#processTransaction(amount, method) { /* transaction logic */ }
#getConfirmation() { return "Payment successful"; }
}
```

2. 🛡️ Encapsulation

📖 Definition: Bundling data and methods together and controlling access to them.

🎯 Example:

```
class BankAccount {
  #balance = 0; // Private field

  constructor(accountNumber) {
    this.accountNumber = accountNumber;
  }

  // Public interface
  deposit(amount) {
    if (amount > 0) {
      this.#balance += amount;
      return `Deposited: ${amount}`;
    }
  }

  withdraw(amount) {
    if (amount > 0 && amount <= this.#balance) {
      this.#balance -= amount;
      return `Withdrawn: ${amount}`;
    }
    return "Insufficient funds";
  }

  getBalance() {
    return this.#balance;
  }
}
```

3. 🧬 Inheritance

📖 Definition: Creating new classes based on existing classes, inheriting properties and methods.

🎯 Example: (Already covered in Extending Classes section)

4. 🐼 Polymorphism

📖 Definition: Same interface, different implementations. Objects of different classes can be treated uniformly.

🎯 Example:

```
class Animal {
  makeSound() {
    return "Some generic animal sound";
  }
}

class Dog extends Animal {
  makeSound() {
    return "Woof!";
  }
}

class Cat extends Animal {
  makeSound() {
    return "Meow!";
  }
}

class Bird extends Animal {
  makeSound() {
    return "Chirp!";
  }
}

// Polymorphism in action
const animals = [new Dog(), new Cat(), new Bird()];

animals.forEach(animal => {
  console.log(animal.makeSound()); // Different outputs for same method call
});
```

5. 🛠 Composition

📖 Definition: Building classes by combining simpler objects rather than inheriting from a base class.

🎯 Example:

```
// Components
class Engine {
  start() {
    return "Engine started";
  }
}
```



```

class Wheels {
  roll() {
    return "Wheels rolling";
  }
}

class GPS {
  navigate(destination) {
    return `Navigating to ${destination}`;
  }
}

// Composition
class Car {
  constructor() {
    this.engine = new Engine(); // Has-a relationship
    this.wheels = new Wheels(); // Has-a relationship
    this.gps = new GPS(); // Has-a relationship
  }

  start() {
    return this.engine.start();
  }

  drive() {
    return this.wheels.roll();
  }

  navigate(destination) {
    return this.gps.navigate(destination);
  }
}

const myCar = new Car();
console.log(myCar.start()); // "Engine started"
console.log(myCar.drive()); // "Wheels rolling"
console.log(myCar.navigate("Home")); // "Navigating to Home"

```

► 🗨 Knowledge Check: OOP Concepts

👛 Interview Questions

- ? What are ES6 Classes and how do they differ from function constructors?
- ? Explain private fields in JavaScript classes.
- ? What is the difference between static and instance methods?

- ▶ ? How does inheritance work in JavaScript classes?
 - ▶ ? Explain method overriding vs method hiding in classes.
-

Practice Tasks

Task 1: Basic Class Creation

Create a `Book` class with the following requirements:

- Properties: title, author, pages, isRead (default false)
- Methods: read() (sets isRead to true), getInfo() (returns book details)

▶  Solution

Task 2: Getters and Setters

Create a `Temperature` class with:

- Private field for celsius
- Getter and setter for fahrenheit
- Method to display both temperatures

▶  Solution

Task 3: Static Methods

Create a `MathUtils` class with static methods:

- `add(a, b)` - adds two numbers
- `isEven(num)` - checks if number is even
- `factorial(n)` - calculates factorial

▶  Solution

Task 4: Inheritance and Polymorphism

Create a `Shape` base class and derived classes `Circle` and `Rectangle` :

- Base class: constructor(color), getColor(), abstract getArea()
- Circle: constructor(color, radius), getArea()
- Rectangle: constructor(color, width, height), getArea()

▶  Solution

Task 5: Private Fields and Encapsulation

Create a `BankAccount` class with:

- Private fields: `balance`, `accountNumber`
- Public methods: `deposit()`, `withdraw()`, `getBalance()`
- Validation: prevent negative deposits/withdrawals

►  Solution

Task 6: Composition Pattern

Create a `Computer` class using composition with `CPU`, `Memory`, and `Storage` components:

- Each component should have its own methods
- Computer should delegate operations to components
- Demonstrate "has-a" relationships

►  Solution

Task 7: Advanced Inheritance

Create a vehicle hierarchy:

- `Vehicle` (base): `brand`, `year`, `start()`, `stop()`
- `Car` extends `Vehicle`: `doors`, `accelerate()`
- `ElectricCar` extends `Car`: `batteryLevel`, `charge()`
- Demonstrate method overriding and super calls

►  Solution

Task 8: Class Factory Pattern

Create a factory function that creates different types of employees based on role:

- `Employee` (base class)
- `Developer`, `Manager`, `Designer` (derived classes)
- Factory function to create appropriate employee type

►  Solution

Task 9: Mixin Pattern

Implement mixins to add functionality to classes:

- `Flyable` mixin

- `Swimmable` mixin
- Apply to `Bird`, `Fish`, and `Duck` classes

► 💡 Solution

📄 Task 10: Complete OOP Project

Create a simple Library Management System with:

- `Book` class with private fields
- `Member` class with borrowing history
- `Library` class managing books and members
- Implement all OOP principles

► 💡 Solution

📅 *For the solution of knowledge check and practise set visit my github repo* 📅

[GitHub: Neeraj05042001](#) 🧑

Summary

- ✅ ES6 Classes - Modern way to create objects
- ✅ Class Features - Fields, methods, getters/setters
- ✅ Static Properties - Class-level functionality
- ✅ Private/Public - Proper encapsulation
- ✅ Inheritance - Code reuse and extension
- ✅ OOP Principles - Abstraction, Encapsulation, Inheritance, Polymorphism
- ✅ Advanced Patterns - Composition, Mixins, Factory

📄 Key Takeaways:

1. Classes are blueprints for creating objects
2. Encapsulation protects data integrity
3. Inheritance enables code reuse
4. Polymorphism allows flexible interfaces
5. Composition can be better than inheritance
6. Private fields provide true data hiding

Keep practicing these concepts and you'll become a JavaScript OOP master! 🚀



Let's Connect & Level Up Together

Follow me for daily JavaScript tips, insightful notes, and project-based learning.



X (Twitter)



LinkedIn



Telegram



Threads



Dive into the full notes on GitHub → **40 Days JavaScript**

© 2025 • Crafted with ❤️ by Neeraj

 ***Next: Day - 31: Master JavaScript Prototypes and Object Patterns - Confused to Confident!*** 



Happy JavaScript coding! 

*Happy Coding!   *